Facultad de Ciencias de la Administración
UNER

TRABAJO FINAL INTEGRADOR

PROGRAMACIÓN II - 2026 – 2do cuatrimestre

TECNICATURA UNIVERSITARIA EN DESARROLLO WEB

Objetivos

- Aplicar y reforzar los conceptos adquiridos durante toda la cursada.

Condiciones de Entrega

- Deberá realizarse una entrega grupal, con grupos de no más de 3 (tres) alumnos.
- Se va a brindar como base para el trabajo una carpeta comprimida que contiene un sistema incompleto. La entrega deberá consistir de una carpeta comprimida que contenga los archivos de la base proveída más los archivos y modificaciones solicitados en las consignas de este documento. Se deberá indicar con claridad la sección del código que se corresponde con cada uno de los ejercicios del documento. La carpeta debe ser subida en la sección correspondiente del Campus.
- Se deberá respetar la fecha de entrega, la misma será informada en el campus.
- El Trabajo Práctico será calificado como **Aprobado** o **Desaprobado**.
- Las soluciones del grupo deberán ser de autoría propia. De encontrarse soluciones idénticas entre diferentes grupos, dichos trabajos prácticos serán calificados como **Desaprobado**, lo cual será comunicado en la devolución.
- Las entregas individuales serán calificadas como **Desaprobado**.

EJERCICIOS:

***Nota**: asumir que todos los parámetros recibidos en los métodos correspondientes a los ejercicios del trabajo son del tipo de datos correcto, con excepción de los casos donde se especifique la necesidad de agregar validaciones.*

***Nota:** para iniciar el sistema, se debe ejecutar:*

```
pip install flask
```

```
python -m flask run
```

Se provee como base un servidor http desarrollado con Python y Flask, que utiliza como fuente de datos precargados un archivo con formato JSON. El sistema, una vez esté funcional, debería ser capaz de recuperar los datos del archivo JSON, crear objetos a partir de los mismos, y retornar una representación de dichos objetos como cadena de texto formateada.

Se va trabajar sobre un modelo de zoológico, incluyendo su nombre, animales, y empleados. Se provee el código de las clases correspondientes a los siguientes diagramas

En el archivo animal.py:

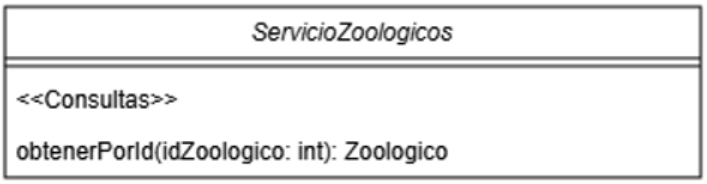
Animal
<<Atributos de instancia>> id:int nombre:string idCuidadorPrincipal: int
<<Constructores>> Animal(id:int, nombre:string, idCuidadorPrincipal:int) <<Comandos>> establecerId(id:int) establecerNombre(nombre:string) establecerIdCuidadorPrincipal(id: int) <<Consultas>> obtenerId(): int obtenerNombre(): string obtenerIdCuidadorPrincipal(): int

En el archivo jirafa.py:

Jirafa(Animal)
<<Atributos de instancia>> largoCuello: int
<<Constructores>> Jirafa(id:int, nombre:string,idCuidadorPrincipal: int,largoCuello: int) <<Comandos>> establecerLargoCuello(largoCuello: int) <<Consultas>> obtenerLargoCuello(): int

Nota: El largo del cuello está medido en centímetros

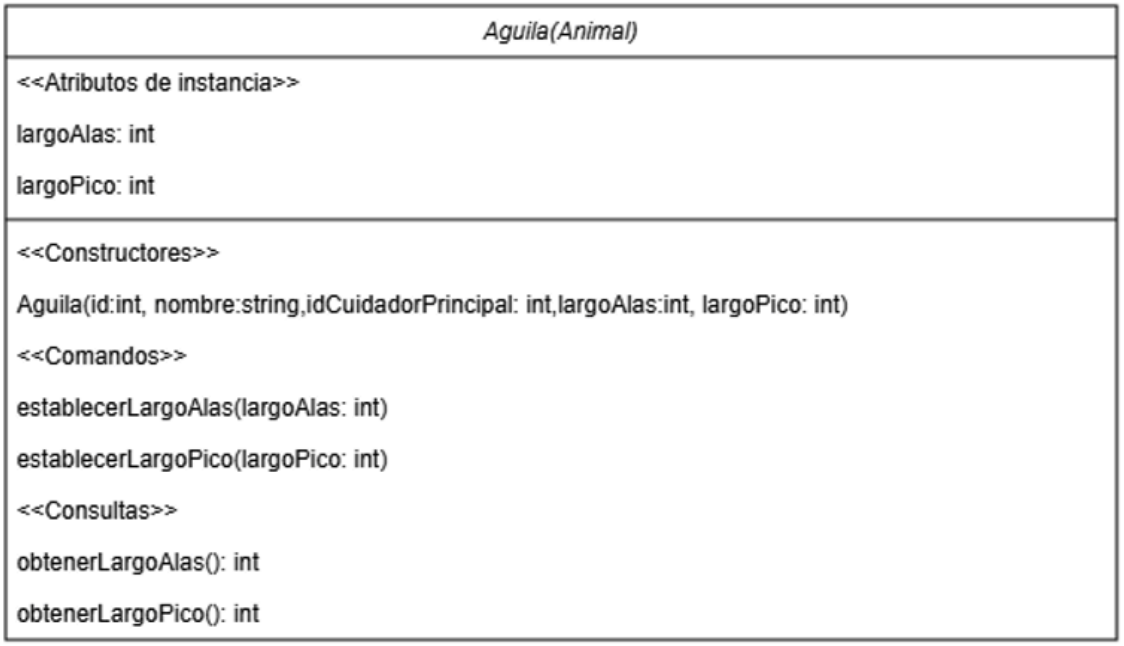
En el archivo servicio_zoologicos.py:



Nota: El método "obtenerPorId" se provee ya implementado, y se encarga de recuperar los datos del archivo JSON correspondientes al zoológico indicado.

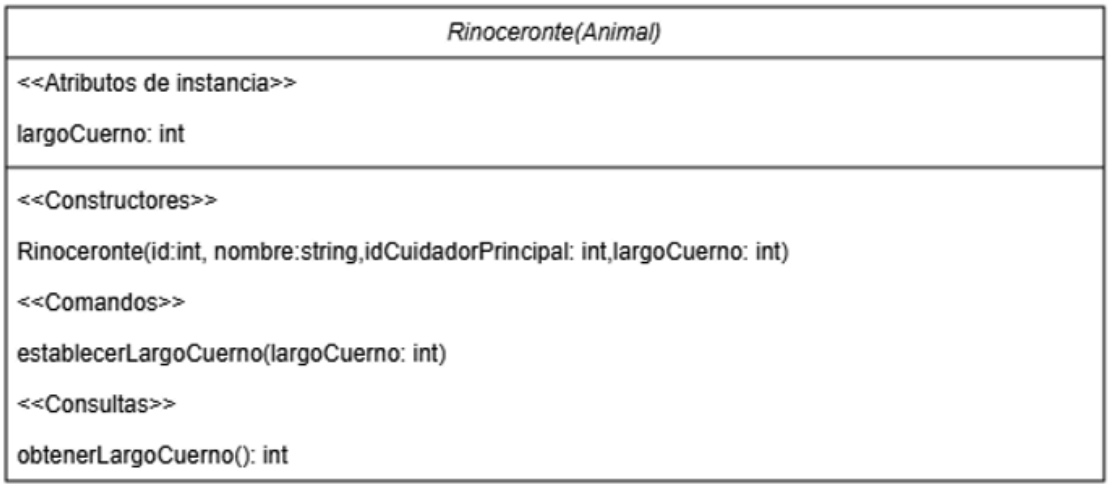
Y se exige que se completen los siguientes ítems:

1) En el archivo aguila.py, generar la clase correspondiente al siguiente diagrama:



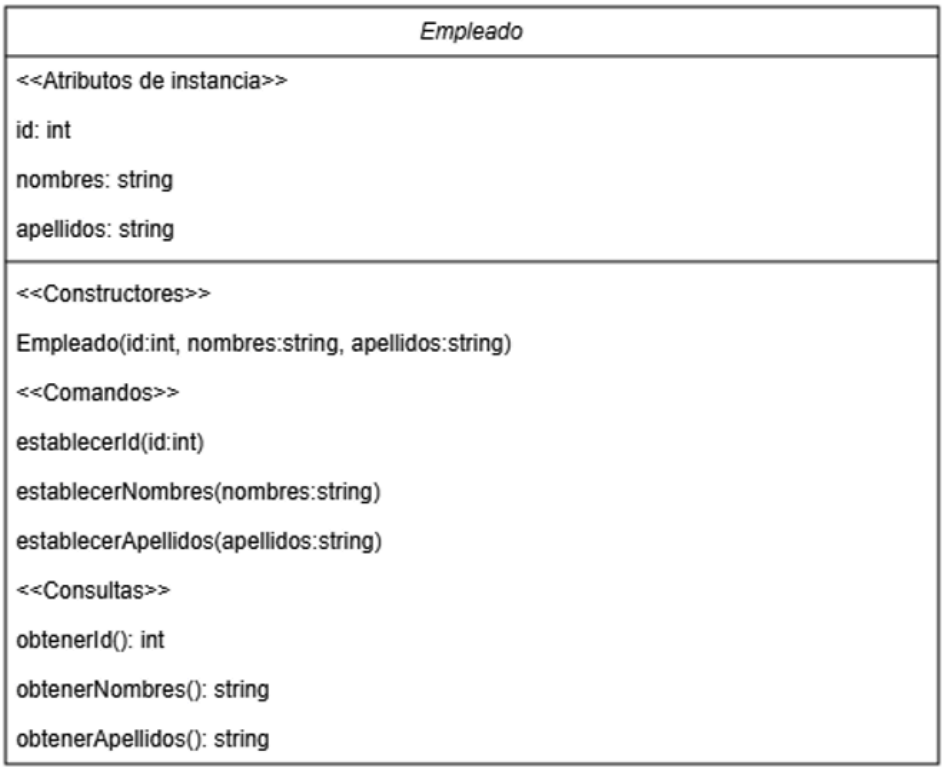
Nota: El largo del pico y las alas está medido en centímetros

2) En el archivo rinoceronte.py, generar la clase correspondiente al siguiente diagrama:

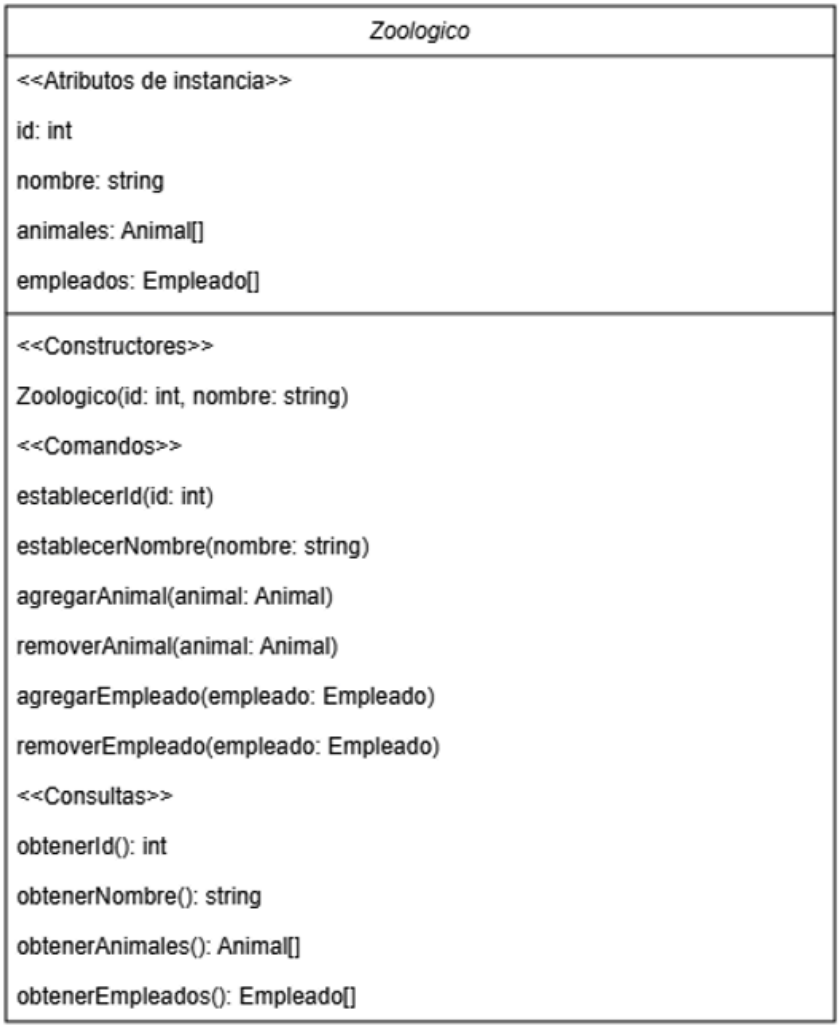


Nota: El largo del cuerno está medido en centímetros

3) En el archivo empleado.py, generar la clase correspondiente al siguiente diagrama:

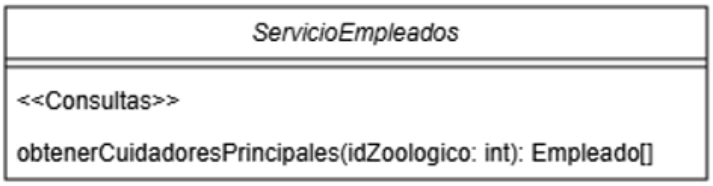


4) En el archivo zoologico.py, generar la clase correspondiente al siguiente diagrama:



5) Actualizar tanto las clases proveídas (con excepción de ServicioZoologicos) como también las clases creadas en los puntos anteriores para incluir el método __eq__, comparando en todos los casos por "id" (con excepción de las subclases de "Animal", que deben heredar __eq__ de la superclase), y el método __str__, mostrando en todos los casos la información correspondiente a todas las propiedades de cada clase (con excepción de la clase "Animal", que declara el método como abstracto para ser implementado en las subclases).

6) En el archivo servicio_empleados.py, implementar el método "obtenerCuidadoresPrincipales" de la clase correspondiente al siguiente diagrama:



La clase y el método se proveen definidos, pero es necesario implementar la lógica interna del método, que se provee con "pass". El mismo debe retornar una lista de los empleados registrados como cuidador principal de al menos un animal en el zoológico seleccionado. Se debe utilizar el método "obtenerPorId" de la clase "ServicioZoologicos" para recuperar todos los datos disponibles del zoológico pertinente, y trabajar sobre los mismos para obtener el resultado solicitado.

7) En el archivo servicio_animales.py, implementar el método "obtenerAnimalesPorCuidadorPrincipal" de la clase correspondiente al siguiente diagrama:



La clase y el método se proveen definidos, pero es necesario implementar la lógica interna del método, que se provee con "pass". El mismo debe retornar una lista de todos los animales que tienen registrado como cuidador principal el cuidador indicado dentro del zoológico indicado. Se debe utilizar el método "obtenerPorId" de la clase "ServicioZoologicos" para recuperar todos los datos disponibles del zoológico pertinente, y trabajar sobre los mismos para obtener el resultado solicitado.

Enlaces para verificación de correctitud:

Para auxiliar el proceso de verificación de correctitud del funcionamiento del sistema, pueden hacer click a los siguientes enlaces dentro de un navegador:

http://localhost:5000/zoologicos/1/

El mismo retorna el resultado de ejecutar str() sobre el zoológico con id igual a 1 (el único registrado en el archivo JSON proveído). Se deberían visualizar todos los datos registrados del zoológico, incluyendo animales, y empleados, cada uno con todos sus datos correspondientes.

http://localhost:5000/zoologicos/1/cuidadores-principales/

El mismo retorna como resultado información de todos los cuidadores principales en el zoológico con id igual a 1.

http://localhost:5000/zoologicos/1/cuidadores-principales/1/animales

El mismo retorna como resultado información de todos los animales dentro del zoológico con id igual a 1 cuyo cuidador principal tenga id igual a 1.